The Journal of Open Source Software

RegularizedOptimization.jl: A Julia framework for regularized and nonsmooth optimization

Youssef Diouane [1], **Maxence Gollier** [1¶], **Mohamed Laghdaf Habiboullah** [1¶], **and Dominique Orban** [1]

1 GERAD and Department of Mathematics and Industrial Engineering, Polytechnique Montréal, QC, Canada ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 03 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

RegularizedOptimization.jl is a Julia package that implements families of quadratic regularization and trust-region methods for solving the nonsmooth optimization problem

$$\underset{x\in\mathbb{R}^n}{\text{minimize}} \quad f(x) + h(x) \quad \text{subject to} \quad c(x) = 0, \tag{1}$$

where $f : \mathbb{R}^n \to \mathbb{R}$ and $c : \mathbb{R}^n \to \mathbb{R}^m$ are continuously differentiable, and $h : \mathbb{R}^n \to \mathbb{R}\cup\{+\infty\}$ is lower semi-continuous. The nonsmooth objective h can be a *regularizer* such as a sparsity-inducing penalty, model simple constraints such as x belonging to a simple convex set, or be a combination of both. All f, h and c can be nonconvex. Together with the companion library ShiftedProximalOperators.jl described below, RegularizedOptimization.jl provides a modular and extensible framework for solving (1), and developing novel solvers. Currently, the following solvers are implemented:

- **Trust-region solvers TR and TRDH** (Aravkin et al., 2022; Leconte & Orban, 2025)
- **Quadratic regularization solvers R2, R2DH and R2N** (Aravkin et al., 2022; Diouane, Habiboullah, et al., 2024)
- **Levenberg-Marquardt solvers LM and LMTR** (Aravkin et al., 2024) used when f is a least-squares residual
- **Augmented Lagrangian solver AL** (De Marchi et al., 2023).

All solvers rely on first derivatives of f and c, and optionally on their second derivatives in the form of Hessian-vector products. If second derivatives are not available or too costly to compute, quasi-Newton approximations can be used. In addition, the proximal mapping of the nonsmooth part h, or adequate models thereof, must be evaluated. At each iteration, a step is computed by solving a subproblem of the form (1) inexactly, in which f, h, and c are replaced with appropriate models about the current iterate. The solvers R2, R2DH and TRDH are particularly well suited to solve the subproblems, though they are general enough to solve (1). All solvers have a non-monotone mode that enhance performance in practice on certain problems (Diouane, Habiboullah, et al., 2024; Leconte & Orban, 2025). All are implemented in an in-place fashion, so that re-solves incur no allocations. To illustrate our claim of extensibility, a first version of the AL solver was implemented and submitted by an external contributor.

Statement of need

Model-based framework for nonsmooth methods

In Julia, (1) can be solved using ProximalAlgorithms.jl, which implements splitting schemes and line-search–based methods (Stella et al., 2017; Themelis et al., 2018). Among others, the

PANOC (Stella et al., 2017) solver takes a step along a direction d, which depends on the gradient of f modified by a L-BFGS Quasi-Newton approximation, followed by proximal steps on the nonsmooth part h.

By contrast, RegularizedOptimization.jl focuses on model-based trust-region and quadratic regularization methods, which typically require fewer evaluations of f and its gradient than first-order line search methods, at the expense of more evaluations of proximal operators (Aravkin et al., 2022). However, each proximal computation is inexpensive for numerous commonly used choices of h, such as separable penalties and bound constraints (see examples below), so that the overall approach is efficient for large-scale problems.

When computing a step by (approximately) minimizing a model, ShiftedProximalOperators.jl implements efficient allocation-free shifted proximal mappings. Specifically, it supports shifted proximal operators of the form

$$\operatorname*{arg\,min}_{t \in \mathbb{R}^n} \tfrac{1}{2}\|t - q\|_2^2 + \nu\psi(t + s; x) + \chi(s + t \mid \Delta\mathbb{B})$$

where q is given, x and s are fixed shifts, $\chi(\cdot \mid \Delta\mathbb{B})$ is the indicator of a ball of radius $\Delta > 0$ defined by a certain norm, and $\psi(\cdot; x)$ is a model of h about x. It is common to set $\psi(t + s; x) = h(x + s + t)$.

These shifted operators allow to (i) incorporate bound or trust-region constraints via the indicator, which is required for the **TR** and **TRDH** solvers, and (ii) evaluate the above in place, without additional allocations, which is currently not possible with ProximalOperators.jl.

RegularizedOptimization.jl provides a consistent API to formulate optimization problems and apply different solvers. It integrates seamlessly with the JuliaSmoothOptimizers (Migot et al., 2021) ecosystem, an academic organization for nonlinear optimization software development, testing, and benchmarking.

The smooth objective f can be defined via NLPModels.jl (Orban et al., 2020b), which provides a standardized Julia API for representing nonlinear programming (NLP) problems. Large collections of such problems are available in CUTEst.jl (Orban et al., 2020a) and OptimizationProblems.jl (Migot et al., 2023), but a use can easily interface or model their own smooth objective.

The nonsmooth term h can be modeled using ProximalOperators.jl, which provides a broad collection of regularizers and indicators of simple sets.

With f and h modeled as discussed above, the companion package RegularizedProblems.jl provides a straightforward way to pair them into a *Regularized Nonlinear Programming Model*

```
reg_nlp = RegularizedNLPModel(f, h)
```

They can also be paired into a *Regularized Nonlinear Least Squares Model* if $f(x) = \frac{1}{2}\|F(x)\|^2$ for some residual $F : \mathbb{R}^n \to \mathbb{R}^m$, as would be the case with the **LM** and **LMTR** solvers.

```
reg_nls = RegularizedNLSModel(f, h)
```

RegularizedProblems.jl also provides a set of instances commonly used in data science and in the nonsmooth optimization literature, where several choices of f can be paired with various nonsmooth terms h. This design makes for a convenient source of reproducible problem instances for testing and benchmarking the solvers in RegularizedOptimization.jl.

Support for both exact and approximate Hessian

In contrast with ProximalAlgorithms.jl, RegularizedOptimization.jl, methods such as **R2N** and **TR** methods support exact Hessians as well as several Hessian approximations of f. Hessian–vector products $v \mapsto Hv$ can be obtained via automatic differentiation through ADNLPModels.jl or implemented manually. Limited-memory and diagonal quasi-Newton

approximations can be selected from LinearOperators.jl. This design allows solvers to exploit second-order information without explicitly forming dense or sparse Hessians, which is often expensive in time and memory, particularly at large scale.

Testing and documentation

The package includes a comprehensive suite of unit tests that cover all functionalities, ensuring reliability and correctness. Extensive documentation is provided, including a user guide, API reference, and examples to help users get started quickly. Documentation is built using Documenter.jl.

Application

A novel implementation of the exact penalty approach (Diouane, Gollier, et al., 2024) for equality-constrained smooth optimization is being developed based on RegularizedOptimization.jl. In it, $h(x) = \|c(x)\|$ and the model $\psi(\cdot; x)$ differs from h itself. Specifically, $\psi(\cdot; x)$ is the norm of a linearization of c about x. This is not covered in the current version of ProximalAlgorithms.jl.

Examples

We illustrate the capabilities of RegularizedOptimization.jl on two nonsmooth and nonconvex problems:

- **Support Vector Machine (SVM) with $\ell_{1/2}^{1/2}$ penalty** for image classification (Aravkin et al., 2024).

- **Nonnegative Matrix Factorization (NNMF) with ℓ_0 penalty and bound constraints** (Kim & Park, 2008).

Below is a condensed example showing how to define and solve SVM problem, and perform a solve followed by a re-solve:

```
using LinearAlgebra, Random, ProximalOperators
using NLPModels, RegularizedProblems, RegularizedOptimization
using MLDatasets

Random.seed!(1234)
model, nls_model, _ = RegularizedProblems.svm_train_model()  # Build SVM model
f = LSR1Model(model)                                         # L-SR1 Hessian approximation
λ = 1.0                                                      # Regularization parameter
h = RootNormLhalf(λ)                                        # Nonsmooth term
reg_nlp = RegularizedNLPModel(f, h)                          # Regularized problem
solver = R2NSolver(reg_nlp)                                  # Choose solver
stats  = RegularizedExecutionStats(reg_nlp)
solve!(solver, reg_nlp, stats; atol=1e-4, rtol=1e-4, verbose=1, sub_kwargs=(max_iter=200,))
solve!(solver, reg_nlp, stats; atol=1e-5, rtol=1e-5, verbose=1, sub_kwargs=(max_iter=200,))
```

The NNMF problem can be set up in a similar fashion:

```
Random.seed!(1234)
m, n, k = 100, 50, 5
model, nls_model, _, selected = nnmf_model(m, n, k)          # Build NNMF model
x0 = rand(model.meta.nvar)                                   # Initial point
λ = norm(grad(model, rand(model.meta.nvar)), Inf) / 200      # Regularization parameter
h = NormL0(λ)                                                # Nonsmooth term
```

DRAFT

```
reg_nls = RegularizedNLSModel(nls_model, h)                    # Regularized problem for LM
solver = LMSolver(reg_nls)                                     # Choose solver
```

Numerical results

We compare **TR**, **R2N**, **LM** and **LMTR** from our library.

We report the following solver statistics in the table: the convergence status of each solver, the number of evaluations of f, the number of evaluations of ∇f, the number of proximal operator evaluations, the elapsed time in seconds and the final objective value. On the SVM and NNMF problems, we use limited-memory SR1 and BFGS Hessian approximations, respectively. The subproblem solver is **R2**.

Method	**Status**	$t(s)$	$\#f$	$\#\nabla f$	$\#prox$	**Objective**
TR (LSR1, SVM)	first_order	3.9349	347	291	4037	179.837
R2N (LSR1, SVM)	first_order	1.9511	185	101	27932	192.493
LM (SVM)	first_order	19.7826	6	2876	1001	201.186
LMTR (SVM)	first_order	12.4967	11	1614	432	188.274
TR (LBFGS, NNMF)	first_order	0.1014	42	40	3160	976.06
R2N (LBFGS, NNMF)	first_order	0.4913	169	107	17789	411.727
LM (NNMF)	first_order	0.1157	14	7042	2601	131.184
LMTR (NNMF)	first_order	0.0697	9	4066	1435	131.186

- Note that for the **LM** and **LMTR** solvers, gradient evaluations count $\#\nabla f$ equals the number of Jacobian–vector and adjoint-Jacobian–vector products.

All methods successfully reduced the optimality measure below the specified tolerance of 10^{-4}, and thus converged to an approximate first-order stationary point. Note that, the final objective values differ due to the nonconvexity of the problems.

- **SVM with $\ell^{1/2}$ penalty: R2N** is the fastest, requiring the fewest gradient evaluations compared to all the other solvers. However, it requires more proximal evaluations, but these are inexpensive. **LMTR** and **LM** require the fewest function evaluations, but incur many Jacobian–vector products, and are the slowest. Note that here, **LMTR** achieves the lowest objective value.
- **NNMF with constrained ℓ_0 penalty: LMTR** is the fastest, and requires a fewer number of function evaluations than all the other solvers. Followed by **TR** which is the second fastest and requires the fewest gradient evaluations, however it achieves the highest objective value. Note that both **LMTR** and **LM** achieve the lowest objective value.

Additional tests (e.g., other regularizers, constraint types, and scaling dimensions) have also been conducted, and a full benchmarking campaign is currently underway.

Conclusion

The experiments highlight the effectiveness of the solvers implemented in RegularizedOptimization.jl.

In ongoing research, the package will be extended with algorithms that enable to reduce the number of proximal evaluations, especially when the proximal mapping of h is expensive to compute.

Acknowledgements

The authors would like to thank Alberto Demarchi for his implementation of the Augmented Lagrangian solver. Mohamed Laghdaf Habiboullah is supported by an excellence FRQNT grant.

DRAFT

Youssef Diouane, Maxence Gollier and Dominique Orban are partially supported by an NSERC Discovery Grant.

References

Aravkin, A. Y., Baraldi, R., & Orban, D. (2022). A proximal quasi-Newton trust-region method for nonsmooth regularized optimization. *Siopt*, *32*(2), 900–929. https://doi.org/10.1137/21M1409536

Aravkin, A. Y., Baraldi, R., & Orban, D. (2024). A Levenberg–Marquardt method for nonsmooth regularized least squares. *Sisc*, *46*(4), A2557–A2581. https://doi.org/10.1137/22M1538971

De Marchi, A., Jia, X., Kanzow, C., & Mehlitz, P. (2023). Constrained composite optimization and augmented Lagrangian methods. *Mathematical Programming*, *201*(1), 863–896. https://doi.org/10.1007/s10107-022-01922-4

Diouane, Y., Gollier, M., & Orban, D. (2024). *A nonsmooth exact penalty method for equality-constrained optimization: Complexity and implementation* (Cahier G-2024-65). GERAD. https://doi.org/10.13140/RG.2.2.16095.47527

Diouane, Y., Habiboullah, M. L., & Orban, D. (2024). *A proximal modified quasi-newton method for nonsmooth regularized optimization* (Cahier G-2024-64). GERAD. https://doi.org/10.48550/arxiv.2409.19428

Kim, J., & Park, H. (2008). *Sparse nonnegative matrix factorization for clustering* (GT-CSE-08-01). Georgia Inst. of Technology. http://hdl.handle.net/1853/20058

Leconte, G., & Orban, D. (2025). The indefinite proximal gradient method. *Coap*, *91*(2), 861–903. https://doi.org/10.1007/s10589-024-00604-5

Migot, T., Orban, D., & Siqueira, A. S. (2021). *The JuliaSmoothOptimizers ecosystem for linear and nonlinear optimization*. https://doi.org/10.5281/zenodo.2655082

Migot, T., Orban, D., & Siqueira, A. S. (2023). *OptimizationProblems.jl: A collection of optimization problems in julia*. https://doi.org/10.5281/zenodo.3672094

Orban, D., Siqueira, A. S., & contributors. (2020a). *CUTEst.jl: Julia's CUTEst interface*. https://doi.org/10.5281/zenodo.1188851

Orban, D., Siqueira, A. S., & contributors. (2020b). *NLPModels.jl: Data structures for optimization models*. https://doi.org/10.5281/zenodo.2558627

Stella, L., Themelis, A., Sopasakis, P., & Patrinos, P. (2017). A simple and efficient algorithm for nonlinear model predictive control. *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, 1939–1944. https://doi.org/10.1109/CDC.2017.8263933

Themelis, A., Stella, L., & Patrinos, P. (2018). Forward-backward envelope for the sum of two nonconvex functions: Further properties and nonmonotone line seach algorithms. *Siopt*, *28*(3), 2274–2303. https://doi.org/10.1137/16M1080240

DRAFT